

How to Build Agents to Generate Kernels for Faster LLMs (and Other Models!)

NeurIPS 2025

Sina Rafati <i>AMD</i>	Hao Li <i>AMD</i>	Azalia Mirhoseini <i>Stanford</i>	Anna Goldie <i>Google Deepmind</i>
Laurence Moroney <i>Arm</i>	Vartika Singh <i>Nvidia</i>	Mark Saroufim <i>Meta</i>	Chris Lattner <i>Modular AI</i>
Sitao Huang <i>UC Irvine</i>	David Kanter <i>ML Commons</i>	Simon Guo <i>Stanford</i>	Kesavan Ramakrishnan <i>Stanford</i>
Vincent Ouyang <i>AMD</i>	Efthimios Gianitsos <i>AMD</i>	Nithya Manohar <i>AMD</i>	Sharon Zhou <i>AMD</i>

Abstract

Graphics processing units (GPUs) power modern deep learning workloads, and the efficiency of GPU kernels directly impacts training throughput, inference latency and energy consumption. Writing optimized kernels requires intimate knowledge of GPU architecture, memory hierarchy, occupancy trade-offs and low-level programming models. This tutorial presents a comprehensive review of GPU kernel optimization, covering three complementary approaches: manual optimization using Heterogeneous-computing Interface for Portability (HIP)/ Compute Unified Device Architecture (CUDA), high-level domain-specific languages like Triton, and emerging large-language-model (LLM) driven agentic systems. We introduce GPU architecture fundamentals, explain memory hierarchy and profiling techniques, and demonstrate how engineers write and optimize kernels by hand. We then survey high-level tools that abstract low-level details while maintaining performance and examine agentic systems that automatically generate and optimize kernels using compiler feedback and profiling data. The tutorial provides detailed hands-on learning paths through three complete tutorial collections covering manual optimization, Triton-based development and agentic kernel synthesis.

Keywords: GPU kernels, kernel optimization, agentic AI, LLMs, Triton, HIP, CUDA, machine learning systems

1 Introduction

1.1 Motivation and Scope

GPUs have become the primary computational substrate for artificial intelligence, enabling the training and deployment of large-scale neural networks. Each GPU contains multiple streaming compute units (CUs) on AMD devices or multiprocessors (SMs) on Nvidia devices that execute thousands of threads in parallel [1]. A kernel is a function executed in parallel across many GPU threads; efficient kernels can reduce training time from days to hours and dramatically lower inference latency [2]. For instance, the attention mechanism in transformer models runs millions of times during training, and a two-fold speedup in the attention kernel directly halves total training time.

Despite its importance, GPU kernel optimization remains challenging. Performance depends on subtle interactions between thread scheduling, memory hierarchy, synchronization and hardware-specific characteristics [2]. Small changes in tiling strategies, loop unrolling or memory alignment can produce order-of-magnitude effects on runtime. Historically, optimization relied on expert engineers who understood GPU microarchitecture and iteratively refined kernels using profilers. Recent work demonstrates that large language models can generate and iteratively optimize kernels using profiling feedback [3, 4], but understanding the underlying system remains essential for guiding and evaluating such agents.

1.2 Three Complementary Approaches

This tutorial covers three approaches to GPU kernel development, each with distinct trade-offs.

Manual optimization (HIP/CUDA) provides complete control over thread mapping, memory layout, synchronization and instruction selection. HIP is AMD’s programming interface while CUDA is Nvidia’s; both provide similar capabilities for direct hardware control. Engineers directly manage shared memory, warp scheduling and occupancy. This approach yields the highest performance but requires significant expertise and development time.

High-level domain-specific languages (Triton) abstract low-level details, allowing programmers to describe computations at the block level while compilers handle memory layout, synchronization and vectorization [5]. Triton kernels are often within 80-95% of hand-tuned HIP/CUDA performance while requiring an order of magnitude less development time.

Agentic AI systems leverage large language models to automatically generate, profile and refine kernels. A typical agentic workflow follows patterns like ‘Understand Intent’ → ‘Make a Plan’ → ‘Use Tools’ → ‘Reflect and Update’. In the case of kernel generation and refinement, this is implemented through the generate-validate-analyze-optimize loop. An LLM analyzes the requirements and formulates a plan to generate code. Utilizing this plan code is generated and then compilers serve as a tool to provide feedback. The compiler results are then reflected upon to continually update the plan and improve overall performance [2, 4]. Agentic systems widen access to kernel optimization but currently struggle with complex kernels and precise resource management.

1.3 Contribution and Organization

This tutorial makes the following contributions:

1. An educational review of GPU architecture, memory hierarchy and profiling tools relevant to kernel optimization
2. A practical guide to manual kernel development, illustrated with real code examples and performance measurements

3. An introduction to high-level tools like Triton, showing how they simplify common patterns while maintaining performance
4. A survey of emerging agentic frameworks (GEAK, STARK, GPU Kernel Scientist, TritonGym) and their design patterns
5. Complete hands-on learning paths connecting theory to three tutorial collections: `hip/` (manual optimization), `triton/` (Triton DSL) and `agentic/` (AI-assisted development)

2 GPU Foundations

2.1 GPU Architecture Overview

Modern GPUs are built from many parallel processing units called compute units (CUs) on AMD hardware or streaming multiprocessors (SMs) on Nvidia hardware. Each SM acts as a self-contained processor featuring arithmetic logic units for general computation, specialized tensor cores for accelerated matrix operations, special-function units for transcendental operations, thousands of registers for fast thread-local storage, configurable shared memory/L1 cache for inter-thread communication, and load/store units for memory access [1]. These SMs connect to a shared L2 cache and off-chip global memory through a high-bandwidth interconnect, forming a two-level memory hierarchy.

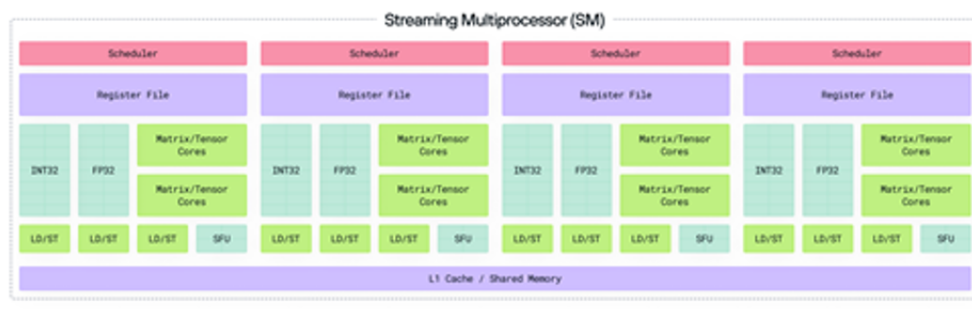


Figure 1: High-level architecture of a streaming multiprocessor (SM) [1].

The GPU programming model organizes computation hierarchically. At the top level, a grid represents the entire parallel workload. Each grid contains multiple thread blocks (called work-groups on AMD), and each block is assigned to a single SM for execution. Within a block, threads are grouped into wavefronts (AMD, 64 threads) or warps (Nvidia, 32 threads) [1]. All threads in a warp execute the same instruction in lockstep on each cycle. This Single Instruction Multiple Thread (SIMT) execution model delivers high throughput for uniform workloads but can cause performance degradation when threads diverge—if a branch causes threads to take different paths, the hardware must serialize execution for each path.

2.2 Memory Hierarchy

Understanding GPU memory is critical because data movement, not computation, typically limits performance. Modern GPUs can execute trillions of floating-point operations per second, but their ability to utilize this throughput depends on how efficiently kernels manage data across the memory hierarchy. GPUs expose several memory spaces that trade capacity for speed—from tiny ultra-fast registers to massive but slower global memory.

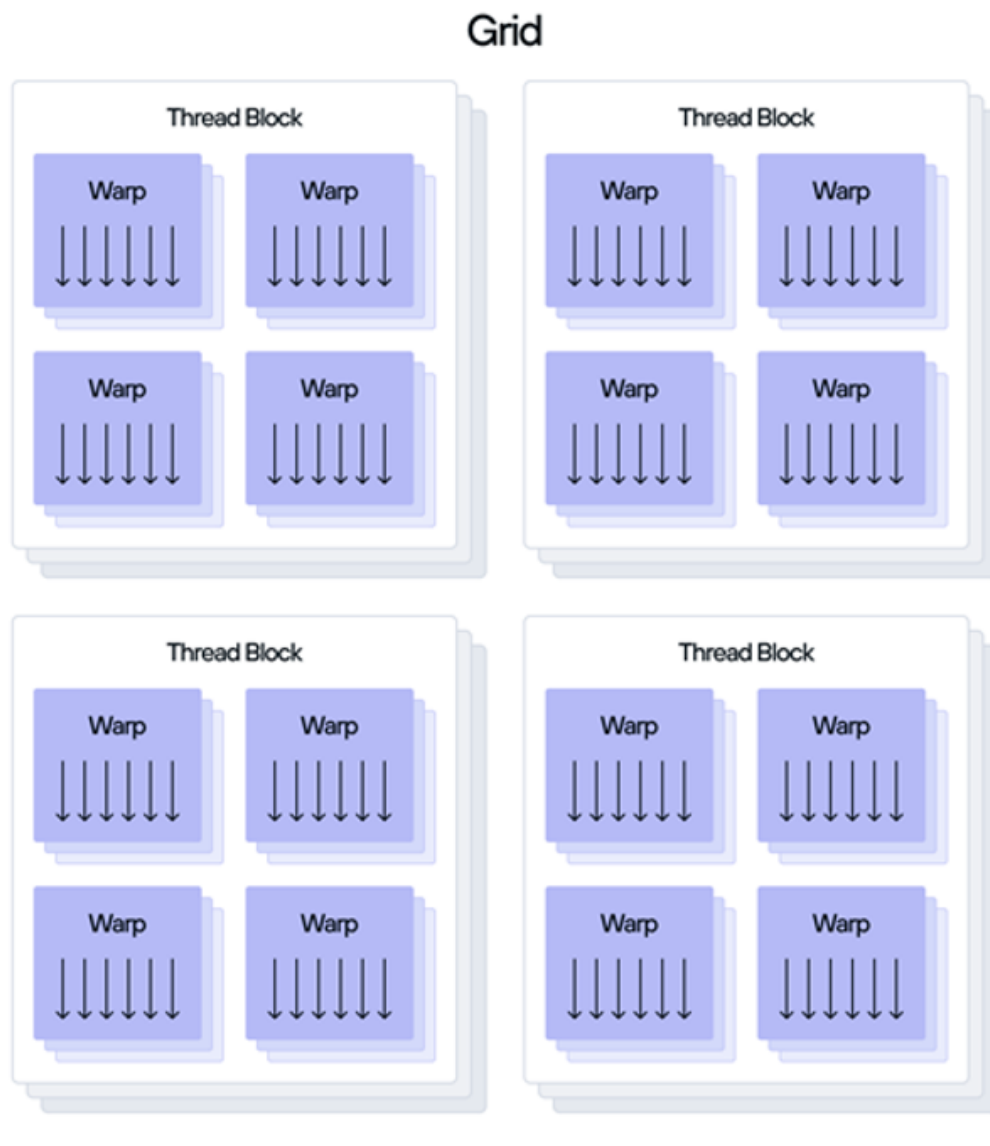


Figure 2: Hierarchy of threads running on a GPU, showing the relationship of the grid, thread blocks, warps, and individual threads, based on HIP Programming Guide [1].

Registers are the fastest storage tier, providing sub-nanosecond access to thread-private data. Each thread has exclusive access to its registers, where scalar variables and frequently reused values reside [6]. Maximizing register usage reduces pressure on slower memory tiers, but excessive register consumption limits occupancy.

Global memory is the largest but slowest memory space, residing off-chip in high-bandwidth DRAM. The host allocates global memory, all threads can access it, and it serves as the primary interface for CPU-GPU data exchange. While modern accelerators deliver 3-4 TB/s of global memory bandwidth, latency remains hundreds of cycles [6]. Coalesced access patterns—where adjacent threads access adjacent memory addresses—are essential to achieving peak bandwidth.

Shared memory (called Local Data Share on AMD) provides on-chip collaborative storage for threads within a block. With latency roughly up to $100\times$ lower than global memory, shared memory enables efficient data reuse and inter-thread communication. Each SM has limited shared memory (typically 64-128 KB) that competes with L1 cache capacity [1]. Effective kernels tile data through shared memory to amplify arithmetic intensity—the ratio of computation to memory access. This is where bank conflicts generally occur when multiple threads simultaneously access different addresses that map to the same memory bank, forcing serialization and reducing throughput.

Occupancy measures resource utilization as the ratio of active warps to the maximum warps an SM can support. Higher occupancy gives the scheduler more warps to switch between, hiding memory latency when one warp stalls. However, occupancy is not a universal goal: compute-bound kernels already saturate execution units regardless of occupancy, while memory-bound kernels with excessive occupancy may thrash caches and degrade performance [7]. The hardware limits occupancy based on register usage, shared memory allocation and thread block size—a fundamental tension in kernel design.

2.2.1 Memory Scopes and Atomic Operations

Memory scopes and atomic operations are critical for correct inter-thread synchronization and coordination, especially in reduction operations, histograms, and cooperative algorithms.

Memory scopes define the visibility domain of memory operations:

- **Thread scope (workitem):** Memory operations only visible to the executing thread. No synchronization with other threads.
- **Block scope (workgroup):** Operations visible to all threads within the same thread block. Used for intra-block cooperation via shared memory.
- **Device scope (agent):** Operations visible across all threads on the GPU. Required for inter-block coordination via global memory.
- **System scope:** Operations visible to both GPU and CPU. Necessary for heterogeneous computing patterns where host and device share data structures.

Memory ordering and fences control when memory operations become visible:

- **Relaxed ordering:** No ordering guarantees; allows maximum reordering for performance.
- **Acquire semantics:** All memory operations after an acquire must see operations before the corresponding release.
- **Release semantics:** All memory operations before a release are visible to threads performing acquire.
- **Acquire-release:** Combines both; commonly used for producer-consumer patterns.

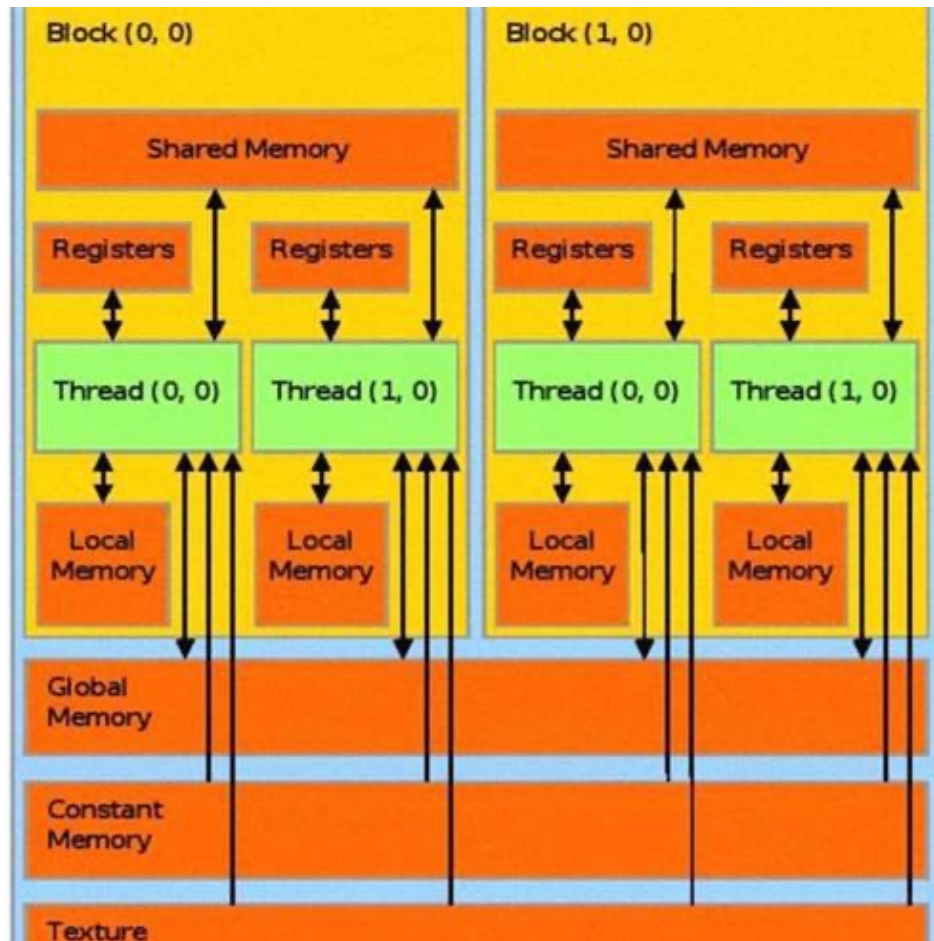


Figure 3: GPU memory model showing the hierarchy from registers to global memory [1].

- **Memory fences:** `__threadfence()`, `__threadfence_block()`, `__threadfence_system()` are explicit barriers that ensure memory operations complete before proceeding.

Atomic operations provide hardware-supported read-modify-write primitives that execute without interference from other threads. Common operations include `atomicAdd()`, `atomicMax()`, `atomicMin()`, `atomicCAS()` (compare-and-swap), and `atomicExch()`. Atomics are essential for:

- Reductions across blocks: Computing global sums, maxima, or counts when results from multiple blocks must be combined.
- Histograms: Multiple threads incrementing bins without explicit locks.
- Dynamic work allocation: Threads claiming work items from a shared queue.

Performance considerations for atomics:

- **Contention:** When many threads target the same memory location, atomics serialize, destroying parallelism. Privatization strategies (per-block accumulators reduced in a second pass) mitigate this.
- **Scope selection:** Use narrower scopes (`__threadfence_block()`) when possible; system-wide fences are expensive.
- **Memory location:** Atomics on shared memory are 10-100× faster than global memory atomics. Tile data to shared memory for high-contention scenarios.

2.3 Profiling and Optimization Tools

Optimization begins with measurement. Profiling tools transform opaque kernel execution into quantifiable metrics that expose bottlenecks and reveal optimization opportunities. Rather than guessing which changes might improve performance, profilers provide empirical evidence about where time is spent, which resources limit throughput, and how efficiently the hardware is utilized.

Profiling metrics fall into several diagnostic categories that guide different optimization strategies:

Time-domain metrics capture when and how long kernels execute. Wall-clock duration establishes baseline performance, while timeline traces reveal kernel launch overhead, host-device synchronization costs, and opportunities for concurrent execution. Comparing kernel duration against theoretical minimum (problem size divided by peak bandwidth or compute) quantifies efficiency gaps.

Throughput metrics measure achieved bandwidth (GB/s) for memory-bound kernels or computational throughput (TFLOPS) for compute-bound kernels. The ratio of achieved-to-theoretical throughput immediately classifies kernels: <30% suggests fundamental inefficiency requiring algorithmic changes, 30-70% indicates room for optimization, and >70% means the kernel approaches hardware limits.

Resource utilization metrics quantify how the kernel consumes SM capacity. Register counts per thread, shared memory bytes per block, and achieved occupancy percentage reveal whether resource constraints limit parallel execution.

Memory system metrics diagnose data movement efficiency. Transaction counts, cache hit rates, and memory unit stall cycles expose whether the kernel achieves coalesced access or suffers from scattered reads. Detailed counters distinguish L1/L2 cache behavior, global memory throughput, and whether loads or stores dominate traffic.

Execution metrics reveal instruction-level behavior. Warp divergence percentages quantify control flow efficiency—high divergence indicates that threads take different branches, serializing execution. Instruction mix ratios (e.g., memory ops per arithmetic op) characterize compute intensity and suggest whether the kernel is compute-bound or memory-bound.

2.3.1 Cross-Vendor Profiling Tools Comparison

Different GPU vendors provide specialized profiling tools optimized for their hardware. Understanding the capabilities and usage patterns of each tool enables effective optimization regardless of target platform. Table 1 summarizes the key features of each tool.

Table 1: Cross-Vendor Profiling Tools Comparison

Tool	Key Features	Strengths	Vendor
AMD rocprof	Per-kernel timing, resource usage, hardware counters, trace modes (HIP/HSA)	Lightweight, scriptable, automated workflows	AMD
Nvidia Nsight Compute	Comprehensive metrics, roofline analysis, source correlation, memory analysis	Rich GUI, automated analysis, optimization guidance	Nvidia
Nvidia Nsight Systems	System-wide timeline, CPU/GPU activity, API traces, multi-GPU support	System-level view, concurrency opportunities	Nvidia

AMD rocprof (ROCm-Profiler) [8]:

- **Basic usage:** `rocprof -stats ./kernel_binary` provides per-kernel timing, resource usage (registers, LDS), and occupancy
- **Custom counters:** `rocprof -i metrics.txt ./kernel_binary` collects hardware counters (L2 hit rate, wavefront occupancy, vector ALU utilization)
- **Trace modes:** `-hip-trace` and `-hsa-trace` capture API calls, kernel launches, and memory transfers
- **Output formats:** CSV for counter data, JSON for timeline traces (visualize with Perfetto or Chrome Tracing)
- **Strengths:** Lightweight, scriptable, integrates well with automated workflows
- **Limitations:** No GUI by default; limited source-level correlation; requires manual metric interpretation

Nvidia Nsight Compute:

- **Basic usage:** `ncu -set full ./kernel_binary` collects comprehensive metrics
- **Roofline analysis:** Automatically plots achieved performance against theoretical compute/bandwidth ceilings
- **Source correlation:** Shows which source lines cause stalls, cache misses, or divergence
- **Memory analysis:** Detailed breakdown of L1/L2/global memory traffic, transaction efficiency, and coalescing effectiveness
- **Warp state analysis:** Visualizes warp scheduler behavior, showing when warps stall and why

- **Strengths:** Rich GUI; extensive automated analysis; source-level insights; built-in optimization guidance
- **Limitations:** Higher overhead; more complex setup; primarily Nvidia-focused metrics

Nvidia Nsight Systems:

- **Basic usage:** `nsys profile ./application` captures system-wide timeline
- **Timeline view:** Interactive visualization of CPU/GPU activity, kernel launches, memory copies, and synchronization points
- **API traces:** Captures CUDA API calls, driver operations, and library calls (cuBLAS, cuDNN)
- **Multi-GPU support:** Profiles distributed workloads across multiple GPUs
- **Strengths:** System-level view; identifies launch overhead, idle time, and concurrency opportunities
- **Limitations:** Less detailed per-kernel metrics compared to Nsight Compute

Workflow recommendation:

1. **Initial profiling:** Use `rocprof -hip-trace` (AMD) or Nsight Systems (Nvidia) to identify which kernels dominate runtime
2. **Kernel deep-dive:** Use `rocprof -i metrics.txt` (AMD) or Nsight Compute (Nvidia) to diagnose specific kernel bottlenecks
3. **Iterative refinement:** Reprofile after each optimization to validate improvements and identify next bottleneck

Both ecosystems provide Python APIs (rocprof via CSV parsing, Nsight via pynvml) for automated analysis in agentic workflows.

The profiling-driven optimization cycle typically proceeds in phases. Initial profiling establishes baseline performance and identifies the primary bottleneck (memory bandwidth, compute throughput, or latency). Subsequent optimizations target that bottleneck—restructuring memory access patterns, adjusting tile sizes, or reducing register pressure—then reprofiling validates improvements and reveals the next limiting factor. This iterative diagnosis-optimization-validation loop transforms kernel tuning from trial-and-error into systematic engineering. Table 2 provides a systematic mapping from common profiling metrics to their corresponding optimization actions.

3 Kernels and Manual Kernel Engineering

3.1 What is a Kernel?

A kernel is a function executed in parallel on the GPU. In both HIP (AMD) and CUDA (Nvidia), the `__global__` qualifier identifies such functions; they are launched from the host but execute on the device [9]. Each thread receives a unique ID used to compute memory addresses and control decisions. Thread blocks (or work-groups in HIP terminology) are mapped to CUs (AMD) or SMs (Nvidia), and each CU/SM divides its block into wavefronts (AMD, 64 threads) or warps (Nvidia, 32 threads). Kernel launches specify an execution configuration (grid and block dimensions) determining thread count and grouping [9].

Table 2: Runtime Metrics to Optimization Action Mapping

Metric	Symptom	Optimization Action
Bandwidth Utilization	<30% of peak	Improve memory coalescing, reduce strided access, use vectorized loads
Occupancy	<50%	Reduce register usage, reduce shared memory, increase block size
L2 Cache Hit Rate	<60%	Improve data locality, tile computation, re-order operations
Warp Divergence	>20%	Restructure conditionals, use branchless operations, partition data
Register Spilling	Any spills detected	Decrease tile size, reduce loop unrolling, use compiler flags
Bank Conflicts	High conflict count	Pad shared memory arrays, use strided access patterns
Atomic Contention	High serialization	Use privatization, reduce to shared memory first, warp-level reductions

3.2 Writing Kernels by Hand

Manual kernel development follows a typical workflow:

1. **Parallel decomposition:** Identify data dimensions and choose grid and block sizes covering the workload. Blocks should be multiples of warp size to avoid partially filled warps [1].
2. **Memory management:** Load data from global memory into registers or shared memory for reuse. Minimize global memory accesses and ensure coalesced access patterns—adjacent threads accessing adjacent addresses to maximize bandwidth.
3. **Occupancy tuning:** Balance register usage and shared memory allocation to maximize active warps [7].
4. **Synchronization:** Use barrier functions (`__syncthreads()` in both HIP and CUDA) to ensure threads complete data loading before proceeding. Avoid warp divergence by minimizing divergent branches within warps [1].
5. **Profiling and iteration:** Compile with appropriate compilers (`hipcc` for AMD or `nvcc` for Nvidia) and profile using `rocprof` (AMD) or Nsight Compute (Nvidia). Examine execution time, bandwidth, occupancy and register usage, then iteratively refine.

3.3 Optimization Patterns

Common optimization patterns include:

Memory coalescing: Adjacent threads access adjacent memory addresses, enabling the GPU to merge accesses into fewer transactions. Strided or scattered access patterns waste bandwidth. Memory-coalescing module demonstrates that coalesced vector addition achieves 3,702 GB/s (116% of peak) versus 196 GB/s (6%) for strided access on AMD MI300X, showing an 18.9× performance improvement.

Tiling and shared memory: Divide data into tiles that fit in shared memory. Threads cooperatively load tiles, synchronize, perform computation using fast shared memory, then write results. Matrix multiplication

benefits dramatically: naive global-memory-only implementations achieve 3.7 TFLOPS, while tiled versions with shared memory reach 13.4 TFLOPS ($3.6\times$ speedup). Note that MI300X’s large cache makes naive versions more competitive than on older GPUs.

Reduction patterns: Sum, max or other associative operations across threads require careful synchronization. Techniques include tree reductions within blocks, atomic operations across blocks, and warp-level primitives for intra-warp reductions.

Bank conflict avoidance: Arrange shared memory layout so threads access different banks. Padding arrays or using strided patterns avoids conflicts [1]. Comparison against naive implementation demonstrates how strided access patterns create bank conflicts. On AMD MI300X, $\text{stride}=32$ causes $1.75\times$ slowdown, while consecutive access provides $1.68\times$ speedup over the worst case. Note that modern AMD GPUs have significantly improved bank conflict handling compared to older architectures, though consecutive access patterns remain best practice for optimal performance.

Loop unrolling: Loop unrolling reduces loop-control overhead and exposes more instruction-level parallelism (ILP), giving the compiler and GPU scheduler additional opportunities to pipeline memory operations and arithmetic instructions. While the absolute TFLOP/s is inflated due to the kernel’s tiny workload and timing overhead, the relative gain clearly shows how unrolling removes loop bookkeeping, increases ILP, and improves GPU saturation. In real workloads, the unroll factor must be chosen carefully to balance ILP against register pressure and occupancy in order to avoid over-unrolling.

4 Agentic Kernel Synthesis

4.1 Motivation: Why Use LLMs for Kernel Optimization?

Manual kernel optimization, as detailed in Section 3, demands deep expertise across multiple domains: GPU architecture, memory hierarchies, programming models, and profiling tools. Engineers must understand how thread mapping affects memory coalescing, how register usage constrains occupancy, which synchronization patterns avoid deadlocks, and how hardware-specific features like tensor cores can be exploited. Even experienced practitioners require days or weeks to optimize complex kernels, and the knowledge doesn’t transfer easily.

Large language models offer a compelling alternative by encoding broad patterns from thousands of lines of GPU code encountered during pre-training. LLMs can propose optimization strategies (tiling, vectorization, memory layout transformations) without requiring engineers to memorize architecture-specific details. They translate high-level intents (“optimize this matrix multiplication for MI300X”) into concrete implementations, suggest fixes for compilation errors, and reason about trade-offs when profiling reveals bottlenecks. Crucially, LLMs democratize kernel optimization: researchers and practitioners without deep GPU expertise can leverage agentic systems to generate efficient kernels, freeing them to focus on algorithmic innovation rather than low-level tuning.

However, LLMs alone are insufficient. They hallucinate hardware details, generate syntactically correct but semantically broken code, and struggle with precise resource management. The solution is agentic systems that combine LLMs with compilers, profilers and evaluators in a feedback loop—transforming code generation from a one-shot guess into an iterative optimization process guided by empirical measurement.

Yet even agentic frameworks are ultimately limited by the quality of their underlying language models. Current general-purpose LLMs, while broadly capable across many domains, lack deep specialization in GPU programming. They may propose memory layouts that violate alignment requirements, suggest tile sizes that exceed shared memory capacity, or recommend optimizations incompatible with specific GPU architectures. These fundamental errors force agentic systems to spend many iterations debugging basic mistakes rather than exploring high-level optimization strategies. To fully realize the potential of agentic kernel synthesis, the community needs GPU-specialized foundation models trained extensively on kernel

code repositories, profiling traces, compiler diagnostics, hardware documentation and optimization case studies. Such models would internalize the semantic relationships between code patterns and performance outcomes, recognize architecture-specific idioms (AMD wavefront programming versus Nvidia warp primitives, RDNA cache hierarchies versus Ampere tensor core scheduling), and generate more correct initial candidates.

4.2 Agentic Loops and Tool Integration

Agentic kernel synthesis follows a continuous optimization cycle: Generate $\rightarrow$ Compile $\rightarrow$ Execute $\rightarrow$ Profile $\rightarrow$ Analyze $\rightarrow$ Regenerate. The LLM agent writes kernel code based on a specification and optimization hints. The compiler checks syntax, reports resource usage (registers, shared memory), and produces warnings about potential inefficiencies. Execution validates correctness against test cases. Profiling measures performance metrics—bandwidth, throughput, occupancy, divergence—as described in Section 2.3. The agent analyzes this feedback, identifies bottlenecks, proposes modifications and generates a refined kernel [3, 4].

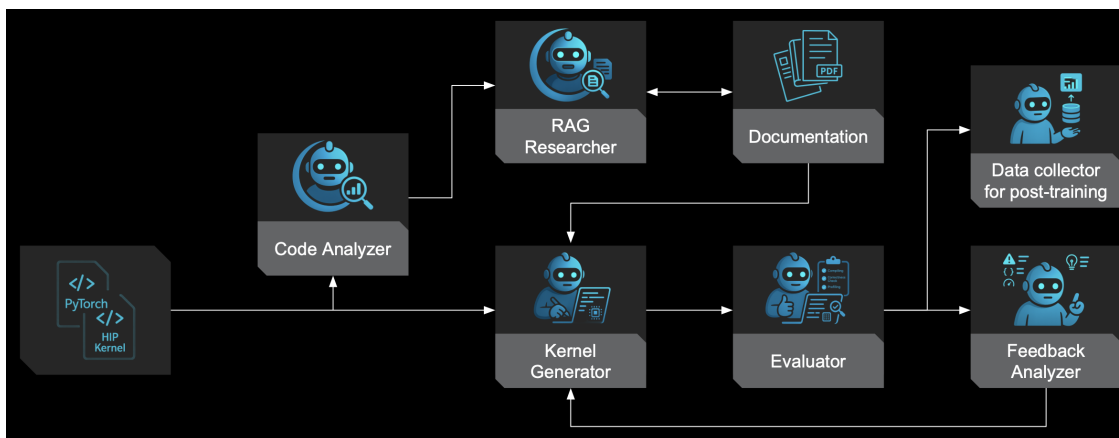


Figure 4: Agentic kernel optimization loop showing the continuous cycle of generation, validation, profiling, and refinement.

This architecture treats compilers and profilers as sensors that ground LLM reasoning in empirical reality. The GPU Kernel Scientist framework demonstrates this approach by using timing data alone to guide iterative refinement, compensating for limited hardware documentation through LLM-encoded knowledge [3]. STARK (Strategic Team of Agents for Refining Kernels) extends this paradigm with multi-agent collaboration, where specialized agents handle planning, implementation and reflection, guided by profiling feedback [2]. TritonGym standardizes the agent-tool interface through function-call APIs, enabling fair comparison of different agentic designs [5].

The key insight is that measurement enables learning. Without feedback, LLMs produce one-shot code that may or may not be efficient. With profiling feedback, agents learn which optimizations work for specific hardware and problem sizes, adjusting strategies across iterations until performance converges to near-optimal solutions.

4.3 Multi-Agent Systems

STARK employs a multi-agent architecture with specialized roles: a planner agent designs optimization strategies, an implementer agent writes code, and a reflection agent analyzes results and adjusts plans [2]. Tree-structured memory enables backtracking when optimizations fail. Grounded instructions provide

hardware-specific hints (tile sizes, vectorization opportunities) to guide generation. Evaluated on KernelBench, STARK produces correct solutions where single-agent baselines often fail and achieves speedups up to $16\times$ over naive implementations [2].

4.4 Prompt Patterns and Knowledge Injection

Effective prompts encode hardware hints such as tile sizes, vector widths, unroll factors and memory scopes. GEAK (Generating Efficient AI-centric Kernels) finds similar Triton code and includes it in prompts, improving correctness and performance [4]. Knowledge injection adds low-level optimization principles and hardware specifications. Reflexion provides the LLM with error traces from failed compilations or incorrect outputs, enabling self-correction [4]. GEAK achieves 54.89% functional correctness on TritonBench and 63.33% on ROCm benchmarks, with average speedups of $2.59\times$ [4].

4.5 Self-Reflection and Memory

Advanced agentic systems incorporate self-reflection: after generating and profiling a kernel, the agent analyzes its own decision-making process and adjusts strategies. For example, if increasing tile size improved performance in three consecutive iterations, the agent encodes this as a heuristic for future kernels. Memory systems store optimization histories across sessions, enabling transfer learning from prior tasks.

5 High-Level Tools: Triton DSL

5.1 Triton Philosophy

Triton is a domain-specific language embedded in Python that simplifies GPU programming while maintaining performance [10]. Unlike HIP (AMD) and CUDA (Nvidia) where programmers manage individual threads, Triton operates at the block level: programmers describe operations on tiles of data, and the compiler handles tiling, memory layout, vectorization and synchronization. Triton kernels are often 90-95% of hand-tuned HIP/CUDA performance but require an order of magnitude less development time.

5.2 Triton vs. HIP/CUDA

A vector addition kernel in HIP or CUDA requires explicit thread indexing, boundary checks and launch configuration. The equivalent Triton kernel uses `tl.program_id` to identify blocks, `tl.arange` to create offsets, and `tl.load/store` with automatic masking for boundary conditions. Both achieve identical bandwidth, but Triton code is more concise and easier to modify.

5.3 Triton for ML Operations

Triton excels at expressing custom ML operations in high-level Python code. While PyTorch’s native operations (LayerNorm, Softmax, MatMul) leverage highly-optimized vendor libraries and are $2\text{-}5\times$ faster than basic Triton implementations, Triton enables rapid development of custom fused kernels and novel operations not available in PyTorch. For actual performance on standard operations, PyTorch natives consistently outperform basic Triton code on AMD MI300X with PyTorch 2.8.0 and Triton 3.4.0.

5.4 Integration with PyTorch

Triton kernels integrate seamlessly with PyTorch via custom autograd functions. Developers define forward and backward passes, register save-for-backward tensors, and call Triton kernels with appropriate grid

configurations. The `triton/` tutorials demonstrate integration patterns for convolution, attention and normalization operators.

5.5 Triton Analysis and Autotuning

Triton provides several tools for understanding performance characteristics and automatically optimizing kernel configurations. These tools transform Triton from a high-level DSL into a complete optimization framework.

Triton Intermediate Representation (IR) and Compilation Pipeline:

Triton compiles Python code through multiple stages, each revealing optimization opportunities:

1. **Triton IR:** High-level representation preserving block-level semantics. View with `MLIR_ENABLE_DUMP=1` environment variable.
2. **LLVM IR:** Lower-level representation after memory layout and synchronization decisions. Inspect with `-dump-llvm-ir` flag.
3. **PTX/GCN assembly:** Final hardware-specific assembly. Use `-dump-asm` to examine instruction counts, register usage, and memory access patterns.

Analyzing compilation artifacts:

```
# Dump Triton IR and assembly
import os
os.environ['TRITON_INTERPRET'] = '0'
os.environ['MLIR_ENABLE_DUMP'] = '1'

@triton.jit
def my_kernel(x_ptr, y_ptr, BLOCK_SIZE: tl.constexpr):
    # kernel code...
    pass

# Launch to trigger compilation
my_kernel[(grid_size,)](x, y, BLOCK_SIZE=1024)
# IR and assembly appear in stdout
```

Examining resource usage: Register count affects occupancy (visible in assembly dump). Shared memory is automatically allocated for blocked operations. Instruction mix (ratio of memory to compute instructions) indicates whether kernel is memory-bound or compute-bound.

Triton Autotuning with `@triton.autotune`:

Triton's autotuner exhaustively searches tile size configurations to find optimal performance:

```

@triton.autotune(
    configs=[
        triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 256,
                        'BLOCK_SIZE_K': 64}, num_stages=3, num_warps=8),
        triton.Config({'BLOCK_SIZE_M': 64, 'BLOCK_SIZE_N': 256,
                        'BLOCK_SIZE_K': 32}, num_stages=4, num_warps=4),
        triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 128,
                        'BLOCK_SIZE_K': 32}, num_stages=4, num_warps=4),
    ],
    key=['M', 'N', 'K'], # Autotune per problem size
)
@triton.jit
def matmul_kernel(a_ptr, b_ptr, c_ptr, M, N, K, ...):
    # kernel implementation...
    pass

```

Autotuning workflow: (1) Define configuration space with candidate tile sizes, warps, and pipeline stages. (2) Specify keying dimensions that determine configuration. (3) Triton compiles each config, measures runtime, caches the best per key. (4) Production code automatically uses cached optimal configuration.

Profiling Triton kernels:

Standard profiling tools work with Triton-generated kernels:

```

# AMD: Profile with rocprof
rocprof -stats python my_triton_script.py

# Nvidia: Profile with Nsight Compute
ncu -set full python my_triton_script.py

```

Performance measurement patterns:

```

import torch
import triton

# Warmup to compile and cache
for _ in range(10):
    my_kernel[(grid,)](x, y, z, BLOCK_SIZE=1024)

# Benchmark
start = torch.cuda.Event(enable_timing=True)
end = torch.cuda.Event(enable_timing=True)

start.record()
for _ in range(100):
    my_kernel[(grid,)](x, y, z, BLOCK_SIZE=1024)
end.record()

torch.cuda.synchronize()
elapsed_ms = start.elapsed_time(end) / 100

# Calculate throughput
bytes = x.numel() * x.element_size() * 2 # read + write
bandwidth_gbs = (bytes / 1e9) / (elapsed_ms / 1000)
print(f"Bandwidth: {bandwidth_gbs:.2f} GB/s")

```

6 Case Studies from Tutorials

We have developed three comprehensive tutorial collections demonstrating manual optimization, Triton development and agentic synthesis. Each tutorial includes working code, profiling scripts and assignments.

6.1 Manual Optimization (hip/)

The `hip/` tutorial contains 9 modules with 18 kernels covering memory coalescing, shared memory tiling, bank conflict resolution, reduction patterns, warp-level primitives, histogram optimization, 1D/2D convolution, Flash Attention and quantized operators. These examples use HIP for AMD GPUs, though all techniques apply equally to CUDA on Nvidia GPUs. Table 3 summarizes key performance results.

Table 3: Manual Optimization Performance Results (AMD MI300X)

Optimization	Baseline	Optimized	Speedup
Memory Coalescing	196 GB/s	3,702 GB/s	18.9×
Matrix Multiplication (Tiling)	3.7 TFLOPS	13.4 TFLOPS	3.6×
Reduction (Warp-level)	24 GB/s	1,063 GB/s	44.0×
Bank Conflict Avoidance	0.57× baseline	0.97× baseline	1.68×

6.2 Triton Development (triton/)

The `triton/` tutorial contains 8 modules demonstrating Triton for ML operations: memory coalescing, matrix multiplication, reduction patterns (softmax, layer norm), fused operators, Flash Attention, grouped GEMM, quantization and custom autograd. All modules compare Triton implementations against PyTorch baselines with built-in benchmarks. Table 4 shows validated performance results.

Table 4: Triton vs PyTorch Performance (AMD MI300X, PyTorch 2.8.0, Triton 3.4.0)

Operation	PyTorch	Triton	PyTorch Advantage
Vector Addition	4,056 GB/s	3,742 GB/s	1.08×
LayerNorm (128×512)	0.0076 ms	0.0368 ms	4.84×
Softmax	0.0061 ms	0.0258 ms	4.23×

Key Finding: PyTorch’s native operations (`torch.nn.functional`, `torch.matmul`) are highly optimized using vendor libraries (rocBLAS, rocDNN) and consistently outperform basic Triton implementations. Triton’s value lies in developing custom operations not available in PyTorch, rapid prototyping, and educational purposes—not in replacing well-optimized PyTorch natives.

6.3 Agentic Synthesis (agentic/)

The `agentic/` tutorial implements a multi-component system: kernel generator (LLM-based), validation pipeline (compilation, correctness, performance), feedback analyzer (LLM-based), optimization agent (coordinator) and memory module. The system iteratively generates, tests and refines kernels.

System Architecture:

- **Validation Pipeline:** Compilation checking → Correctness verification → Performance profiling
- **LLM Integration:** OpenAI (GPT-4) and Anthropic (Claude) support
- **Memory System:** Persistent optimization history and learning
- **Optimization Strategies:** Memory coalescing, tiling, vectorization, kernel fusion

Validated Optimization Results (AMD MI300X, Triton 3.4.0):

Table 5 shows optimization results from the agentic synthesis system.

Table 5: Agentic Kernel Optimization Results

Kernel	Problem Size	Optimization	Speedup
Vector Addition	8M elements (32 MB)	Block size 128 → 1024	1.14×

Key Optimization: Increasing block size from 128 → 1024 improved GPU occupancy and memory coalescing, achieving 14% speedup for this memory-bound operation.

7 Discussion and Future Directions

7.1 Current Limitations

Agentic kernel synthesis shows promise but faces challenges:

Correctness: Agents can generate syntactically valid but semantically incorrect code. Robust validation pipelines with comprehensive test suites are essential.

Complex kernels: Agents struggle with operations requiring intricate synchronization (e.g., multi-pass algorithms, producer-consumer patterns).

Hardware diversity: Prompts tuned for one GPU architecture may not transfer to others. Agents need architecture-specific knowledge.

Resource constraints: Agents often fail to precisely manage registers and shared memory, leading to reduced occupancy or compilation failures.

Debugging loops: When agents repeatedly fail to fix compilation errors or performance issues, human intervention becomes necessary.

Kernel fusion opportunities: Identifying which operations should be fused remains challenging. Manual analysis or simple heuristics miss complex fusion patterns that could significantly improve performance.

7.2 Detecting Fusion Candidates

Kernel fusion combines multiple operations into a single kernel, eliminating intermediate memory traffic and reducing launch overhead. Systematic fusion detection identifies opportunities that manual inspection might miss.

Fusion benefits:

- Reduced memory traffic: Intermediate results stay in registers/shared memory instead of round-tripping through global memory
- Lower launch overhead: Single kernel launch instead of multiple successive launches
- Improved arithmetic intensity: More compute per byte transferred
- Better cache utilization: Temporal locality keeps working set in cache

Fusion candidate detection heuristics:

1. Memory-bound chain pattern:

- Pattern: Sequence of operations where each reads previous output, performs lightweight computation, writes result
- Detection: Profile shows high memory traffic (>70% of runtime), low compute utilization (<30% of peak FLOPS)
- Example: `relu(bias_add(matmul(A, B)))` — intermediate activations written then immediately read
- Opportunity: Fuse bias addition and ReLU into matmul output; keep results in registers

2. Single-use intermediate:

- Pattern: Tensor produced by one kernel and consumed by exactly one subsequent kernel, then discarded
- Detection: Data flow analysis shows tensor has single consumer, not reused elsewhere in graph
- Example: Softmax denominator used only in division step
- Opportunity: Fuse producer and consumer; eliminate intermediate tensor allocation

3. Large launch overhead:

- Pattern: Many small kernels with individual runtime < 0.1 ms, where launch overhead dominates
- Detection: Timeline profiling shows kernel execution time $<$ gap between kernel launches
- Example: Sequence of element-wise operations (add, multiply, clamp) on same tensor
- Opportunity: Merge into single fused element-wise kernel

4. Repeated tensor reads:

- Pattern: Multiple kernels reading the same input tensor independently
- Detection: Memory profiler shows same address ranges read by multiple kernels
- Example: Attention mechanism computes Q, K, V from same input; LayerNorm computes mean and variance
- Opportunity: Load shared input once, compute all outputs in single kernel

5. Reduction followed by broadcast:

- Pattern: Reduce tensor along dimension, then broadcast result back for element-wise operation
- Detection: Reduction kernel followed by broadcast read pattern
- Example: LayerNorm (reduce to mean/variance, broadcast for normalization), Softmax (reduce to max/sum)
- Opportunity: Online algorithms that maintain accumulators without materializing reduced values

6. Data layout transformations:

- Pattern: Transpose/reshape followed by compute, or compute followed by transpose
- Detection: Memory profiler shows non-coalesced access patterns
- Example: Transpose $\rightarrow$ MatMul, or MatMul $\rightarrow$ Transpose for NCHW $\leftrightarrow$ NHWC conversions
- Opportunity: Fuse layout transform with compute to perform transformation during read/write

7.3 Promising Directions

GPU-specialized foundation models: As mentioned in Section 4.1, the most impactful advancement would be developing foundation models specifically trained for GPU programming. Pre-training on extensive kernel code repositories, profiling traces, compiler outputs and hardware documentation would enable models to internalize architecture-specific patterns, generate more correct initial code, and reduce iteration counts in agentic loops.

Richer feedback: Integrating detailed hardware counters (cache hit rates, warp occupancy, stall reasons) beyond simple timing metrics could provide more informative signals for agent learning. Coupling profiling data with compiler diagnostics (register pressure warnings, predicted occupancy) would help agents diagnose bottlenecks more accurately.

Hybrid approaches: Combining human-designed algorithmic templates with agentic fine-tuning may balance innovation and correctness. Humans provide high-level structure (tiling strategy, memory layout), while agents optimize parameters (tile sizes, unroll factors).

Multi-modal agents: Agents that reason about performance using both code and profiling visualizations (flame graphs, roofline plots, cache behavior timelines) could make better optimization decisions by connecting visual patterns to code changes.

Formal verification: Integrating formal methods to prove correctness properties (memory safety, race-freedom, absence of bank conflicts) would increase confidence in agentic outputs and eliminate entire classes of bugs before execution.

Curriculum learning: Training agentic systems on progressively complex kernels—starting with simple element-wise operations, advancing through reductions and matrix operations, culminating in multi-stage fusion kernels—could build robust optimization strategies and improve generalization.

7.4 Benchmarks and Evaluation

Rigorous evaluation is essential for advancing agentic kernel synthesis. Unlike traditional software engineering where correctness alone suffices, GPU kernels must simultaneously satisfy functional correctness, performance efficiency, and resource constraints.

Evaluation dimensions for agentic kernel generation:

- **Functional correctness:** Does the generated kernel produce numerically correct outputs across diverse inputs, including edge cases?
- **Performance efficiency:** How close is the kernel’s throughput to theoretical hardware limits?
- **Compilation success rate:** What fraction of generated kernels compile without errors?
- **Optimization iteration count:** How many generate-profile-refine cycles does the agent require?
- **Generalization capability:** Does the agent succeed on out-of-distribution tasks?
- **Resource efficiency:** Does the kernel respect hardware constraints?

Existing benchmarks:

Table 6 compares the major benchmarking frameworks for GPU kernel evaluation.

Table 6: Comparison of GPU Kernel Benchmarking Frameworks

Benchmark	Focus	Key Features	Example Results
TritonGym [5]	Agentic LLM workflows in Triton	Standardized API, maintained operators, OOD tasks	Fair agent comparison
KernelBench [2]	Ground-truth implementations	Diverse operations, reference kernels, test suites	STARK: up to 16× speedup
TritonBench [4]	Triton-specific	ML operators, PyTorch integration	GEAK: 54.89% correctness
ROCmBench [4]	AMD ROCm ecosystem	AMD-specific operators	GEAK: 63.33% correctness

TritonGym [5] is an emerging benchmark specifically designed for agentic LLM workflows in Triton GPU code generation. It standardizes the agent-tool interface through function-call APIs, ensuring fair comparison. TritonGym includes maintained operators, community-contributed samples, out-of-distribution tasks, and DSL extensions.

KernelBench, developed alongside STARK [2], offers a collection of ground-truth kernel implementations spanning diverse operation types: element-wise operations, reductions, matrix operations, convolutions, attention mechanisms, and normalization layers. Each task includes reference implementations and test suites validating correctness.

TritonBench and *ROCmBench*, used in the GEAK evaluation [4], provide operation-specific benchmarks tailored to Triton and AMD’s ROCm ecosystem. GEAK reports both functional correctness rates (54.89% on modified TritonBench, 63.33% on ROCm benchmark) and performance speedups.

7.5 Educational Implications

Understanding GPU architecture and manual optimization remains crucial even as agentic tools improve. Agentic systems should augment expertise rather than replace it. Effective kernel developers need to:

- Understand memory hierarchy to interpret profiling data
- Recognize optimization patterns to guide agent prompts
- Validate agent outputs for correctness and efficiency
- Debug failures when agents produce incorrect code

Hands-on tutorials combining profiling, manual optimization and agentic tools build intuition and prepare practitioners to effectively leverage AI assistance.

7.6 Common Failure Modes and Debugging Checklist

Even experienced GPU programmers encounter recurring failure patterns. This checklist systematically diagnoses and resolves the most common kernel optimization pitfalls. Table 7 provides a quick reference guide.

Table 7: Common GPU Kernel Failure Modes Quick Reference

Category	Failure Mode	Quick Fix
Memory-Related	Uncoalesced access (<30% bandwidth)	Restructure thread indexing, use vectorized loads
	Bank conflicts	Pad shared memory arrays
	Cache thrashing (<60% L2 hit)	Tile to fit cache, reorder ops
Resource Constraints	Register spilling	Reduce tile size, decrease unrolling
	Shared memory exhaustion	Reduce tile dimensions
	Low occupancy (<50%)	Reduce resource usage, increase block size
Synchronization	Race conditions	Insert <code>__syncthreads()</code> , use atomics
	Warp divergence (>20%)	Restructure conditionals
	Deadlocks	Ensure all threads reach barriers
Correctness	Numerical instability	Use Kahan summation, normalize
	Boundary errors	Add proper masking
	Atomic contention	Use privatization strategy

Memory-Related Failures:

1. Uncoalesced Memory Access

- Symptom: Low bandwidth utilization (<30% of peak)
- Fix: Restructure indexing so adjacent threads access adjacent addresses

- Verification: Bandwidth should approach $>80\%$ of theoretical peak

2. Bank Conflicts in Shared Memory

- Symptom: Shared memory operations take longer than expected
- Fix: Pad shared memory arrays; use strided or permuted access patterns
- Verification: Profile should show zero or minimal bank conflicts

3. Cache Thrashing

- Symptom: Low L2 hit rate ($<60\%$)
- Fix: Tile computation to fit working set in cache
- Verification: L2 hit rate improves to $>80\%$

Resource Constraint Failures:

1. Register Spilling

- Symptom: Profiler reports register spills; unexpectedly low occupancy
- Fix: Reduce tile size; decrease loop unrolling; use `-maxrregcount` compiler flag
- Verification: Profiler shows zero spills; occupancy increases

2. Shared Memory Exhaustion

- Symptom: Compilation fails with “insufficient shared memory”
- Fix: Reduce tile dimensions; use smaller data types
- Verification: Kernel compiles; occupancy reaches acceptable levels ($>50\%$)

3. Low Occupancy

- Symptom: GPU underutilized; many idle execution units
- Fix: Reduce resource usage per thread; increase block size
- Verification: Occupancy increases; latency hiding improves

Synchronization and Correctness Failures:

1. Race Conditions

- Symptom: Non-deterministic results; occasional incorrect outputs
- Fix: Insert `__syncthreads()` between dependent phases; use atomic operations
- Verification: Results deterministic and correct across all runs

2. Warp Divergence

- Symptom: High divergence percentage ($>20\%$)
- Fix: Restructure conditionals; use branchless arithmetic
- Verification: Divergence metric drops; runtime improves

3. Deadlocks

- Symptom: Kernel hangs; never completes
- Fix: Ensure all threads in block reach same `__syncthreads()`
- Verification: Kernel completes successfully

Numerical and Correctness Failures:

1. Numerical Instability

- Symptom: Results differ significantly from CPU version; NaN or Inf values
- Fix: Use Kahan summation for large reductions; normalize intermediate values
- Verification: Results match reference within acceptable tolerance (typically $1e-5$ for FP32)

2. Boundary Condition Errors

- Symptom: Segmentation faults; incorrect outputs for non-power-of-two sizes
- Fix: Add proper masking for partial tiles; verify all indexing includes bounds checks
- Verification: No segfaults; correct results for all input sizes

3. Atomics Contention

- Symptom: Kernel much slower than expected despite correct output
- Fix: Privatization (per-block accumulators, then reduce); use shared memory atomics
- Verification: Atomic contention metrics drop; performance improves dramatically

Launch Configuration Failures:

1. Incorrect Grid/Block Dimensions

- Symptom: Not all data processed; results partially correct
- Fix: Calculate grid size: $(N + \text{BLOCK_SIZE} - 1) / \text{BLOCK_SIZE}$
- Verification: All outputs computed; no wasted threads

2. Excessive Kernel Launch Overhead

- Symptom: Many tiny kernels; launch gaps dominate timeline
- Fix: Fuse kernels; use persistent kernels; CUDA/HIP graphs
- Verification: Timeline shows fewer, longer-running kernels

Performance Optimization Failures:

1. Suboptimal Tile Size

- Symptom: Performance significantly below reference implementations
- Fix: Autotune tile sizes across range (16, 32, 64, 128, 256)
- Verification: Performance within 90% of best-known implementation

Debugging Workflow:

1. Isolate the failure: Run minimal test case; bisect to find failing component

2. Check correctness first: Verify outputs before optimizing performance
3. Profile systematically: Collect baseline metrics; identify primary bottleneck
4. Fix one issue at a time: Change single parameter; measure impact; iterate
5. Validate improvements: Ensure fix doesn't break correctness
6. Reprofile: Optimization may expose new bottleneck; repeat cycle

8 Conclusion

This tutorial has presented a comprehensive overview of GPU kernel optimization, spanning manual optimization with HIP/CUDA, high-level domain-specific languages like Triton, and emerging agentic AI systems. We have covered GPU architecture fundamentals, memory hierarchies, profiling techniques, and practical optimization patterns. Through three complete tutorial collections, we provide hands-on learning paths that connect theory to practice.

The field of agentic kernel synthesis is rapidly evolving. While current systems show promise, significant challenges remain in correctness, handling complex kernels, and generalizing across hardware architectures. The most impactful future direction is developing GPU-specialized foundation models that deeply understand hardware-software co-design principles. Combined with richer profiling feedback, hybrid human-agent workflows, and rigorous benchmarking, these advances will democratize GPU kernel optimization and accelerate AI innovation.

Understanding GPU fundamentals remains essential even as automation improves. Effective practitioners must interpret profiling data, guide agent prompts with optimization intuition, and validate outputs for correctness and efficiency. By combining human expertise with AI assistance, we can unlock the full potential of GPU hardware and build faster, more efficient machine learning systems.

References

References

- [1] Modular. Mojo manual – GPU architecture overview. <https://docs.modular.com/mojo/manual/gpu/architecture/>, 2025. Accessed: 2025-12-26.
- [2] Anonymous. STARK: Strategic team of agents for refining kernels. *arXiv preprint*, October 2025. arXiv:2510.16996.
- [3] Anonymous. GPU kernel scientist: An LLM-driven framework for iterative kernel optimization. In *Proceedings of the 42nd International Conference on Machine Learning, ICML '25*, 2025. arXiv:2506.20807.
- [4] AMD ROCm Blog. GEAK: Introducing triton kernel AI agent & evaluation benchmarks. <https://rocm.blogs.amd.com/software-tools-optimization/triton-kernel-ai/README.html>, August 2025. Accessed: 2025-12-26.
- [5] Anonymous. TritonGym: A benchmark for agentic LLM workflows in triton GPU code generation. In *International Conference on Learning Representations*, 2026. Under review.

- [6] Carpentries Incubator. GPU programming: Registers, global, and local memory. https://carpentries-incubator.github.io/lesson-gpu-programming/global_local_memory.html, November 2024. Accessed: 2025-12-26.
- [7] AMD GPUOpen. Occupancy explained. <https://gpuopen.com/learn/occupancy-explained/>, December 2023. Accessed: 2025-12-26.
- [8] HLRS. Introduction to RocProf profiling tool. GPU-AMD Workshop, September 2023. Accessed: 2025-12-26.
- [9] Cornell Virtual Workshop. Understanding GPU architecture > GPU characteristics > kernels and SMs. https://cvw.cac.cornell.edu/gpu-architecture/gpu-characteristics/kernel_sm, 2025. Accessed: 2025-12-26.
- [10] OpenAI. Triton: Open-source GPU programming for neural networks. <https://github.com/openai/triton>, 2023. Accessed: 2025-12-26.